

COMP 364/464

Project 2 Part A: Counting N-Queens Solutions

Term: Fall 2026

Instructor: Melanie Cornelius

Assigned: 10 October 2026

Due: 31 October 2026, 11:59 PM AoE

Description

In Part A you parallelized N-Queens and timed it. In Part B you find out why it runs as fast or as slow as it does. You'll watch your Part A programs from the outside using the two tools from class: the `/proc` filesystem and `perf`.

You don't need to change your Part A code except to add debug symbols at compile time (see below). If your measurements show you something you want to fix, you're welcome to fix it, just report the changes and the motivation for the change, and be sure to resubmit your new code for grading.

Before You Start

For this assignment, Linux is required. `/proc` and `perf` are Linux-only. If you built Part A on macOS, see the document that follows this one for running Linux in a VM.

You are provided a handful of example scripts using `perf` and the `/proc` filesystem, plus several examples relevant to the problems in Project 2 Part B. Please feel free to use them!

Install perf

On Ubuntu or Debian: `sudo apt install linux-tools-common linux-tools-$(uname -r)`.

On Fedora or RHEL: `sudo dnf install perf`.

Check that it works with `perf stat ls`.

Permissions

If `perf` says you don't have permission, run `cat /proc/sys/kernel/perf_event_paranoid`

Make sure it's set to `-1` for completeness.

See https://linux-audit.com/kernel/sysctl/kernel.perf_event_paranoid/ for more information.

`sudo sysctl kernel.perf_event_paranoid=-1` sets it until the next reboot.

VMs and Containers

Hardware counters (cycles, instructions, cache and branch events) may show up as `<not supported>`.

Software events (task-clock, context-switches, cpu-migrations, page-faults) still work. If your counters aren't supported, indicate the lack of coverage in your report.

Debug Symbols

Add `-g` to the `CFLAGS` in your Makefile. With GCC, `-g` lets `perf` show you function names and source lines.

Problem Size

For the `/proc` sections you need runs long enough to watch: pick the `N` where your serial version takes at least 30 seconds (probably 14 or 15). Use this same `N` for everything in Part B unless a section says

otherwise.

Part 1: Using perf stat

For each implementation (serial, OpenMP static, OpenMP dynamic, MPI, and Hybrid if you're in 464), at your long N, and for each worker count you used in Part A, run:

```
perf stat -e
task-clock,cycles:u,instructions:u,branches:u,branch-misses:u,cache-references:u,
cache-misses:u,context-switches,cpu-migrations ./your_program N
```

Record in a table: wall time, task-clock, CPUs utilized, instructions, instructions per cycle (IPC), branch-miss rate, and cache-miss rate.

For MPI, `perf stat` in front of `mpirun` won't give you per-worker values. Put it inside a small wrapper so each worker writes its own file:

```
#!/usr/bin/env bash
# perf_worker.sh: run perf stat on one MPI worker, one output file per worker.
# Open MPI sets OMPI_COMM_WORLD_RANK; MPICH sets PMI_RANK.
W=${OMPI_COMM_WORLD_RANK:-${PMI_RANK:-0}}
exec perf stat -e task-clock,instructions:u,context-switches -o
    perf_worker_${W}.txt "$@"
```

Run it with `mpirun -np 4 ./perf_worker.sh ./your_mpi_program N`. Workers are numbered starting at 0.

In your report, answer the following:

1. As you add workers, does the total instruction count stay about the same? If it grows, where are the extra instructions coming from? (Hint: a thread or worker that is waiting isn't always asleep.)
2. How does "CPUs utilized" compare to the number of workers you asked for? Compare static and dynamic.
3. For MPI: do all workers execute about the same number of instructions? What does that tell you about your task breakdown? Make a bar chart of instructions per worker.
4. N-Queens is a search full of branches. What's your branch-miss rate?
5. What's your cache-miss rate? Use your perf data and the size of your board to explain whether the cache matters for this problem.

Part 2: Flame Graphs with perf record

Record your serial version and one parallel version of your choice, then find where the time goes:

```
perf record -e cycles:u -g -o nq.data ./your_program N
perf report -i nq.data --stdio --no-children --sort symbol
```

Then make a flame graph of each, starting from the class flame graph script (06). Include the SVGs in your submission.

Answer in your report:

1. What fraction of samples are from your own code, and what fraction are from the OpenMP runtime (functions starting with `gomp_` or `GOMP_`) or the MPI library? Report the proportional runtime.
2. Your search is recursive. Describe what the recursion looks like in the flame graph and why.
3. Observer effect: time each recorded program with and without `perf` record. How much does recording slow it down? Do the same for `perf stat`. Which one would you trust for timing, and why?

Part 3: Using `/proc`

3a. *OpenMP threads over time.*

Modify the class `threads` script (03) to sample every 0.5 seconds and write CSV: time, thread ID, state, CPU seconds, core. Run it against your static and dynamic versions with 4 or more threads. Plot CPU seconds over time for each thread, one plot per version.

- If your static version has load imbalance, this is where you'll see it. If you don't see imbalance, explain why your work split doesn't produce any.
- Do threads move between cores? Does that match `cpu-migrations` from Part 1? Try `OMP_PROC_BIND=true` and report what changes.

3b. *Context switches.*

Read `voluntary_ctxt_switches` and `nonvoluntary_ctxt_switches` from `/proc/PID/task/TID/status` for each thread at the end of a run. (The numbers in `/proc/PID/status` are for the main thread only.) Then run your OpenMP version with twice as many threads as you have cores. What happens to each kind of context switch, and to the run time? Connect this to the different kinds of context switches.

3c. *MPI workers*

Each MPI worker is its own process. While your MPI version runs, find the PIDs (`pgrep -f your_mpi_program`) and sample each one's `stat` and `status` every 0.5 seconds: state, CPU seconds, `VmRSS`, context switches. When a worker finishes its tasks and waits for the others, what state is it in? Does that agree with what `perf` indicated in Part 1?

3d. *Memory*

Use the class `RSS` script (04) on your serial version, your OpenMP version, and one MPI worker. N-Queens needs very little memory for the board, so where does the `RSS` come from? Look at `/proc/PID/maps` to find out (libraries, thread stacks, MPI shared memory). Compare the total `RSS` of one OpenMP process with the total across all MPI workers at the same worker count.

3e. *464 only: Hybrid*

While your hybrid version runs, take one snapshot of every thread of every worker: worker number, thread ID, core, state, CPU seconds. Are threads from different workers sharing cores? Are threads from the same worker squeezed onto one core? Look up your MPI's process binding option (Open MPI: `--bind-to`, MPICH: `-bind-to`), try at least two settings, and report what changes in placement and in run time.

Part 4: Connecting It Back to Part A

Pick at least two results from your Part A timings that surprised you or that you couldn't fully explain (plateau'd speedup, schedule performance,, MPI doing worse than expected, etc.). Use evidence from this work to explain the anomaly.

Deliverables:

- All scripts you used to collect data, with a comment at the top saying what it collects
- Raw data: perf output files and CSVs
- Plots and flame graph SVGs
- An updated README explaining how to run your programs and data collection
- A report section answering the questions above, plus the “Connecting It Back” section

Grading:

- Data collection scripts and README: 20%
- perf results and explanations: 30%
- /proc results and explanations: 30%
- Connecting it back to Part A: 20%

The same academic honesty and AI policies apply. If you use AI tools, document your process and model usage. You should be able to explain every number in your report and every line of every script, since the exams cover this material.

Turn in Part B as a .zip or .tar on Sakai.

Running Linux on a Mac or Windows Machine

/proc and perf only exist on Linux. If you're on a Mac, run an Ubuntu virtual machine with Multipass. If you're on Windows, use WSL2 (Windows Subsystem for Linux).

On a Mac (Apple Silicon or Intel)

Install Multipass (needs Homebrew, download the installer from multipass.run):

```
brew install --cask multipass
```

Check how many CPUs your Mac has with `sysctl -n hw.ncpu`, then create the VM. Give it as many CPUs as you can spare (leave at least 2 for macOS):

```
multipass launch 24.04 --name hpc --cpus 6 --memory 8G --disk 20G
```

Share your project folder (comp364 below, change it to whatever you named your directory) with the VM so you can edit on your Mac and build inside the VM:

```
multipass mount ~/comp364 hpc:/home/ubuntu/comp364
```

Open a shell in the VM and install:

```
multipass shell hpc
sudo apt update
sudo apt install -y build-essential git openmpi-bin libopenmpi-dev
linux-tools-common linux-tools-$(uname -r)
```

Anything you save in `/home/ubuntu/comp364` inside the VM shows up in `~/comp364` on your Mac.

When you're done for the day, run `multipass stop hpc`

When you want to run again, run `multipass shell hpc`

On Windows 10 or 11

Open PowerShell as Administrator and run:

```
wsl --install
```

Restart when it asks. After the restart, an Ubuntu window opens and asks you to pick a username and password.

2. In the Ubuntu window, install what you need:

```
sudo apt update
sudo apt install -y build-essential git openmpi-bin libopenmpi-dev
linux-tools-common linux-tools-generic
```

WSL2 runs its own kernel, so the normal `perf` command may complain that it can't find a matching version. Point it at the one you just installed:

```
sudo ln -sf $(ls /usr/lib/linux-tools/*/perf | head -1) /usr/local/bin/perf
```

Keep your project inside your Linux home folder (for example ~/comp364), not under /mnt/c. Building and running from /mnt/c is much slower. To open the folder in Windows Explorer, run explorer.exe . from inside it.

Open “Ubuntu” from the Start menu for subsequent runs.

Windows and Mac:

Check that perf works:

```
sudo sysctl kernel.perf_event_paranoid=-1
perf stat ls
```

You’ll need to rerun the sysctl line each time the VM or WSL restarts.

Expect some hardware counters to be missing. Inside a VM or WSL2, events like cycles, instructions, cache-misses, and branch-misses may show <not supported>. Software events (task-clock, context-switches, cpu-migrations, page-faults) and all of /proc will work. If cycles isn’t supported, use -e cpu-clock instead of -e cycles:u for perf record in Part 2. Note what was missing in your report and answer the hardware-counter questions with “not available on my setup.” **You will not lose points.**

Timing in a VM is noisier than on bare metal. Run each timing at least three times and report the median.